

RecoverTogether

iOS Accountability & Recovery App

README & Technical Documentation

Version 2.0 | April 2026

1. App Overview

RecoverTogether is an iOS application designed to support individuals on their recovery journey, whether from social media, or other harmful time consuming.

The app addresses a critical problem: Refraining from using social media individually is difficult to sustain alone. RecoverTogether remedies this by offering a real-time group support system, in which users keep each other in check and on track.

Problem Statement

Many people struggling with addiction or sobriety lack a reliable support system. Traditional recovery programs offer group meetings, but there is no persistent, real-time digital tool that allows friends and accountability partners to monitor each other's progress, intervene when someone falls off track, and celebrate wins together.

Main Features

- User Onboarding — New users complete profile setup (name, sobriety date, avatar) and add their first scheduled task before accessing the app
- Shared Calendar — Add tasks with specific start/end times. Only you can edit, delete or stop your own tasks. Your fellow group members see your calendar by tapping your name on the home page
- Task Session Monitoring — Start a task with a countdown timer. If you leave the app, group members are notified immediately with an task specific alert, and an option to send a message toward the user who ended the task

- Mark Done — Checkbox marks task complete, stops timer, highlights task green on calendar, and notifies the whole group
- Community Dashboard — Shows all group members with color-coded status dots: green = completed, orange = active, red = away/interrupted
- Member Calendar View — Tap any member's name to see their full calendar with all their tasks and real-time status
- Discord-Style Group Chat — Multiple chat rooms, create groups, add members from friends list, remove members, share invite codes
- Friends System — Search by name or username, send/accept/decline friend requests, remove friends
- Group Management — Create groups with invite codes, join via code, add/remove members, leave or delete groups
- Invite System — Share invite links via iMessage, email, or copy link. Friends join your group automatically
- Focus Sessions — Timed sessions (5-60 min) with motivational quotes rotating every 30 seconds
- Mood Tracker — Daily emoji check-in, trigger chips, weekly bar chart
- Leaderboard — Real Firestore data ranked by streak and completed tasks
- Streak Badges — Dashboard shows streak badges next to member names (3d=🌟, 7d=🔥, 30d=🏆)
- Delete Account — Permanently removes account from Firebase Auth, Firestore, all groups and friend lists
- Profile Management — Edit name, avatar color, sobriety date. Dark/light mode toggle

2. User Guide

Installation

- Clone the repository: <https://github.com/Muazsuhaib/RecoverTogether>. If 404 error go to <https://github.com/Muazsuhaib> and request access.
- Open RecoverTogether.xcodeproj in Xcode 15 or 16 (NOT Xcode 26 beta — Firebase incompatible)
- Add GoogleService-Info.plist to the project root (download from Firebase Console)
- Make sure GoogleService-Info.plist Target Membership is checked for RecoverTogether target
- Add Firebase via File > Add Package Dependencies:
<https://github.com/firebase/firebase-ios-sdk>
- Select FirebaseAuth, FirebaseFirestore, and FirebaseMessaging (version 11.x)
- Connect iPhone or select a simulator, press Cmd + R to build and run

First Launch — New User Flow

- Tap Sign Up and fill in Full Name, Username, Email, and Password
- The setup flow appears automatically — it only shows ONCE for new users
- Step 1: Complete your profile — pick avatar color, enter name and sobriety date
- Step 2: Add your first task — enter title, pick a day, set start and end time (e.g. Thursday 2:00 PM to 4:00 PM)
- Step 3: Join a group — enter an invite code from a friend, or skip to create one later
- Step 4: All set! Tap Go to Dashboard to enter the app
- After logging out and back in, the setup flow does NOT reappear

Core Features Guide

Calendar

- Tap the Calendar tab to view month or week view — only YOUR tasks appear here
- Tap + to add a task with title, date, start time, and end time
- Tap your own task to open the detail sheet with Start and Mark Done buttons
- Start button begins a countdown timer — leaving the app notifies your group
- Mark Done stops the timer, turns the task green, and sends a group notification
- Other users' tasks are NOT shown in your calendar — tap their name on Dashboard to see theirs

Dashboard

- Home tab shows all community members with color-coded status dots
- Green dot = completed a task, Orange = currently active, Red = left during task, Gray = idle
- Status text shows details like Finished: Homework or Working on: Gym
- Tap any member card to view their full weekly calendar
- Tap Invite Friends button to share your group code

Chat

- Chat tab shows all groups you belong to
- Tap + to Create a Group (select friends to add) or Join with Code
- Inside a chat, tap ... menu to View Members, Invite to Group, or Delete Group
- In Members sheet, tap + to add more friends, swipe left to remove a member
- Messages show your username not your email

Friends

- Search by name or username to find users
- Tap Add to send a friend request
- Accept or decline incoming requests with green/red buttons
- Swipe to remove a friend — removes from both sides instantly

Group

- Go to Profile > My Group to create or join a group
- Copy your invite code and share it — friends enter it to join your group
- Tap Share Invite to send via iMessage, email, or copy link

3. Testing Strategy

Testing Approach

The app was tested through manual functional testing on both the iOS Simulator and physical iPhone devices running iOS 18. Multi-user testing used two separate accounts on two devices connected to the same Firebase project to verify real-time synchronization, notification delivery, and group features.

Sample Test Cases

Test Case	Steps	Expected Result	Status
User Sign Up	Tap Sign Up, fill name/username/email/password, tap Create Account	Account created, setup flow appears with profile step	✅ Pass
Setup Flow Once	Complete setup, log out, log back in	Setup flow does NOT appear again on subsequent logins	✅ Pass
Add Calendar Task	Calendar tab, tap +, enter title, date, start/end time, tap Add	Task appears on selected day with correct time block in green	✅ Pass
Owner-Only Task Control	User B tries to delete User A's task	Delete/Start buttons hidden for non-owners. Task protected	✅ Pass
Start Task + Timer	Tap task, tap Start, watch countdown	Timer counts down. Group notified with auto-reply textbox	✅ Pass
Leave App Notification	Start task, switch to another app	Group receives alert: left during task with Send button	✅ Pass
Mark Task Done	Tap task, tap Mark as Done	Task turns green, timer stops, group gets congratulations alert	✅ Pass
15-Min Overdue Alert	Create task at past time, wait 1 min	Group members receive overdue notification with quick reply	✅ Pass

Community Calendar	Dashboard, tap member name	Opens their full calendar showing all their real tasks	✅ Pass
Status Colors	Member starts task / completes / leaves	Orange=active, Green=done, Red=away shown on Dashboard	✅ Pass
Create Group Chat	Chat tab, tap +, Create Group, add friends	Group created with selected friends as members	✅ Pass
Join Group via Code	Chat tab, tap +, Join with Code, enter RCVR-XXXX	User joins the group and sees it in chat list	✅ Pass
Add Members to Chat	Open chat, ..., View Members, tap +	Friends list shown, select friends, tap Add to add them	✅ Pass
Friend Request	Search username, tap Add, other user accepts	Both users appear in each other's friends list	✅ Pass
Remove Friend	Swipe friend in Friends list, tap Remove	Removed from both users' lists instantly	✅ Pass
Delete Account	Profile, tap Delete Account, confirm	Firebase Auth deleted, removed from groups and friend lists	✅ Pass
Leaderboard	Tap Leaderboard in quick access	Shows real group members ranked by streak and tasks done	✅ Pass
Chat Username	Send a message in group chat	Message shows username (e.g. @muaz123) not email	✅ Pass

4. Technology Stack

Hardware Requirements

- Mac computer running macOS 13 Ventura or later for development
- iPhone running iOS 16.0 or later for physical device testing
- Minimum 8GB RAM recommended for Xcode with simulator
- Note: Xcode 26 beta has Firebase compatibility issues — use Xcode 15 or 16 (stable)

Software Requirements

Component	Version / Details
Xcode	15.x or 16.x (stable) — NOT Xcode 26 beta
Swift	5.9
SwiftUI	iOS 16+ framework
iOS Deployment Target	iOS 16.0 minimum
Firebase SDK	11.x via Swift Package Manager
macOS	13.0 Ventura or later

Supported Platforms

- iPhone (iOS 16.0+) — primary platform
- iOS Simulator on Mac — fully supported for development and testing

5. APIs & Third-Party Packages

Firebase (Google)

Integrated via Swift Package Manager from <https://github.com/firebase/firebase-ios-sdk>

Package	Usage
FirebaseAuth	User registration, login, logout, password reset, and permanent account deletion
FirebaseFirestore	Real-time cloud database. Stores users, groups, messages, tasks, activity feed, friend requests, and reminders. Uses addSnapshotListener for real-time updates
FirebaseMessaging	Push notification infrastructure for group alerts and task notifications
FirebaseCore	Core initialization via FirebaseApp.configure() in app entry point

Integration: `FirebaseApp.configure()` is called in `RecoverTogetherApp.swift`. All reads use real-time listeners. Auth state monitored via `addStateDidChangeListener`. Setup completion stored in both Firestore and UserDefaults to prevent race conditions.

Apple Frameworks

- SwiftUI — declarative UI for all views
- UserNotifications — local push notifications for task and group alerts
- Combine — reactive app lifecycle detection (foreground/background)
- UIKit — share sheet (UIActivityViewController), opening URLs for SMS and email

6. Contributors

Name	Role	Contributions
Muaz Fnu	Lead iOS Developer + Fullstack	• Built the complete SwiftUI app from scratch. Implemented all features: authentication, onboarding, calendar with task ownership, session monitoring, notifications with auto-reply, Discord-style chat, friends system, community dashboard with color-coded status, member calendar view, leaderboard with real data, group management, invite system, profile with delete account, focus sessions, mood tracker, and all Firebase integration. • Built the complete SwiftUI frontend including all views, navigation, UI design, and theme system. Implemented Firebase integration (Auth, Firestore, Messaging). • Developed the calendar system with start/end times, task session monitoring, activity detection, real-time notifications, group chat, friends system, onboarding flow, mood tracker, leaderboard, and all app features from concept to completion. • Developed Flask + WebSocket backend components and contributed to the Firestore security rules and data structure design.

Adrian Wu	Backend + Tester	• Set up database schemas, assisted with Firebase Firestore structure, conducted multi-device testing, identified bugs in friend sync, chat usernames, task ownership, and member calendar. Provided detailed improvement feedback throughout development. • Set up the PostgreSQL database schema as a backup solution just in case firebase wasn't working. Assisted with Git version control, branch management, and code review. • Contributed to backend architecture discussions and supported Firebase database structure planning. QA and Documentation. Contributed to testing strategies, documented app features, and assisted with project coordination and presentation preparation. • Was the main tester of the app. Also contributed to assisting Muaz with helping him develop code.
Joshua Saikin	Backend Developer + Leader	• Led backend architecture decisions, recommended Firebase over PostgreSQL, contributed to Firestore security rules and data structure design, assisted with Flask and WebSocket backend components. • Led backend infrastructure decisions, recommended Firebase as the primary database over PostgreSQL. • Acted as the bridge for everyone. • Provided key details and moral support. • 2nd back up developer. Assisted Muaz and Adrian in developing code.

Jack	Backup coder	• Early development and design for user interaction features • Planning for front end implementation (original design using React Native) • Documentation
Matthew	Backup coder	• Early slides design • Updated Documentation per each update • Assisted with providing feedback and making decisions towards the final application.